

The background features several abstract geometric shapes. In the top left, there is a purple circle and a green line forming a corner. Below these, a large, complex shape is composed of overlapping purple, blue, and green segments. To the right of this, a smaller blue shape is visible. In the bottom left, there is an orange circle and a green line forming a corner. In the bottom center, there are three blue curved lines and an orange circle.

Polymorphis m

Flow of Discussion

What is Polymorphism in Python?

- Polymorphism in Built-in function len()

Polymorphism With Inheritance

- Example: Method Overriding
- Override Built-in Functions

Polymorphism In Class methods

- Polymorphism with Function and Objects

Polymorphism In Built-in Methods

Method Overloading

Operator Overloading in Python

- Overloading + operator for custom objects
- Overloading the * Operator
- Magic Methods

Polymorphism



Polymorphism, is the ability of an object to take many forms. In simple words, polymorphism allows us to perform the same action in many different ways.



Polymorphism, a method can **process objects differently depending on the class type or data type**.



Polymorphism, is an object oriented programming concept that refers the ability of a variable, function or object to take on multiple forms.

Why is polymorphism used?

- The **concept of polymorphism** helps you to perform a single action in various ways. It helps you to define one interface and benefit from multiple implementations.

Polymorphism

For example, Jessa acts as an employee when she is at the office. However, when she is at home, she acts like a wife. Also, she represents herself differently in different places. Therefore, the same person takes different forms as per the situation.



Employee Role



Wife Role



Tennis Player Role

Jessa takes different forms as per the situation

Example

```
class Person:

    def __init__(self, name, role):
        self.name = name
        self.role = role

1 usage
    def emp(self):
        print(self.name, "is an", self.role, "when she is in the office")
1 usage
    def wife(self):
        print(self.name, "is an", self.role, "when she is at home")

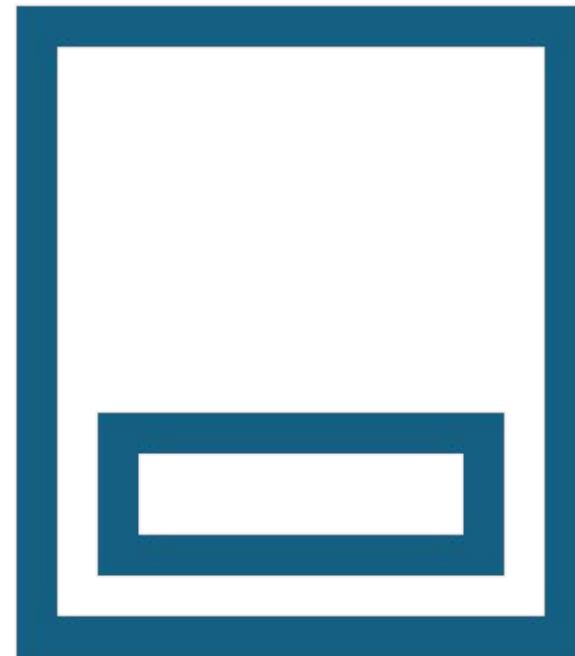
    def player(self):
        print(self.name, "is an", self.role, "when she is at court")

# Person Object
person = Person( name: "Jessa", role: "Wife")
person.emp()
# calls method from a Person class
person = Person( name: "Jessa", role: "Employee")
person.wife()
```

```
Jessa is an Wife when she is in the office
Jessa is an Employee when she is at home
```

Polymorphism in Built-in function len()

- The built-in function len() calculates the length of an object depending upon its type.
- If an object is a string, it returns the count of characters, and If an object is a list, it returns the count of items in a list.
- The len() method treats an object as per its class type.



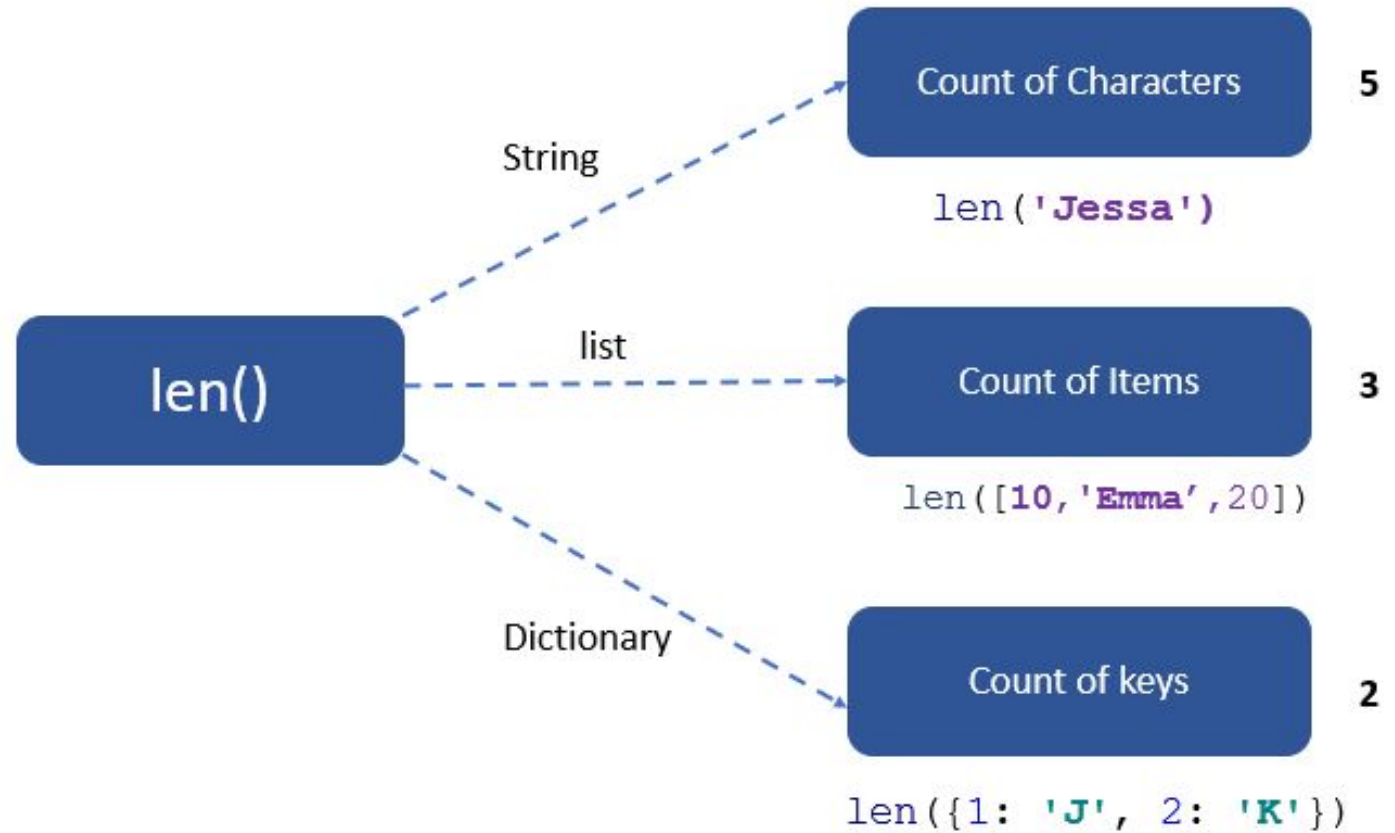
Example

```
students = ['Emma', 'Jessa', 'Kelly']  
school = 'ABC School'  
  
# calculate count  
print(len(students))  
print(len(school))
```

Example:

```
387 # Polymorphism with len() function
388
389 # Using len() with different data types
390 print(len("Hello, World!")) # String
391 print(len([1, 2, 3, 4, 5])) # List
392 print(len((10, 20, 30)))    # Tuple
393 print(len({"name": "Alice", "age": 25})) # Dictionary
394
395 # Using len() with a custom class
396 1 usage
397 class CustomObject:
398     def __init__(self, data):
399         self.data = data
400     def __len__(self):
401         return len(self.data)
402
403 # Creating an instance of CustomObject
404 obj = CustomObject([100, 200, 300, 400])
405
406 # Using len() on the custom object
407 print(len(obj))
408
```

len() Functions Polymorphis m



Polymorphism with Inheritance

- Polymorphism is mainly used with inheritance. In **inheritance**, child class inherits the attributes and methods of a parent class.
- The existing class is called a base class or parent class, and the new class is called a subclass or child class or derived class.

Example

- Through Inheritance, multiple versions of the same class (derived classes) can be created based on the original class (base class).

```
class Animal:
    def make_sound(self):
        print("Some generic animal sound")

class Dog(Animal):
    def make_sound(self):
        print("Bark")

    def eat(self):
        print("Bone")

class Cat(Animal):
    def make_sound(self):
        print("Meow")

    def eat(self):
        print("Fish")

dog = Dog()
cat = Cat()

dog.make_sound()
cat.make_sound()

dog.eat()
cat.eat()
```



Polymorphism with Inheritance using Method Overriding

- Using **method overriding** polymorphism allows us to defines methods in the child class that have the same name as the methods in the parent class.
 - This **process of re-implementing the inherited method in the child class** is known as Method Overriding.
-

Advantage of Method Overriding

- It is effective when we want to extend the functionality by altering the inherited method. Or the method inherited from the parent class doesn't fulfill the need of a child class, so we need to re-implement the same method in the child class in a different way.
- Method overriding is useful when a parent class has multiple child classes, and one of that child class wants to redefine the method. The other child classes can use the parent class method. Due to this, we don't need to modification the parent class code



Example

```
class Vehicle:

    def __init__(self, name, color, price):
        self.name = name
        self.color = color
        self.price = price

    def show(self):
        print('Details:', self.name, self.color, self.price)

    def max_speed(self):
        print('Vehicle max speed is 150')

    def change_gear(self):
        print('Vehicle change 6 gear')

# inherit from vehicle class
class Car(Vehicle):
    def max_speed(self):
        print('Car max speed is 240')

    def change_gear(self):
        print('Car change 7 gear')

# Car Object
car = Car('Car x1', 'Red', 20000)
car.show()
# calls methods from Car class
car.max_speed()
car.change_gear()

# Vehicle Object
vehicle = Vehicle('Truck x1', 'white', 75000)
vehicle.show()
# calls method from a Vehicle class
vehicle.max_speed()
vehicle.change_gear()
```



Override Built-in Functions

- In Python, we can change the default behavior of the built-in functions.

For example, we can change or extend the built-in functions such as `len()`, `abs()`, or `divmod()` by redefining them in our class.

len() Example

- In this example, we will redefine the function len()

```
class Shopping:
    def __init__(self, basket, buyer):
        self.basket = list(basket)
        self.buyer = buyer

    def __len__(self):
        print('Redefine length')
        count = len(self.basket)
        # count total items in a different way
        # pair of shoes and shir+pant
        return count * 2

shopping = Shopping(['Shoes', 'dress'], 'Jessa')
print(len(shopping))
```

```
Redefine length
4
```

abs() Example

- This image show the Absolute Value

```
# Python program to illustrate abs() on integers
num1 = 10
print('Absolute value of 10 is:', abs(num1))

num2 = -38
print('Absolute value of -38 is:', abs(num2))

num3 = 0
print('Absolute value of 0 is:', abs(num3))
```

```
Absolute value of 10 is: 10
Absolute value of -38 is: 38
Absolute value of 0 is: 0
```

divmod() Example

- Quotient Remainder

```
45
46 print('(5, 4) = ', divmod(5, 4))
47 print('(10, 16) = ', divmod(10, 16))
48 print('(11, 11) = ', divmod(11, 11))
49 print('(15, 13) = ', divmod(15, 13))
50
51 # divmod() with int and Floats
52 print('(8.0, 3) = ', divmod(8.0, 3))
53 print('(3, 8.0) = ', divmod(3, 8.0))
54 print('(7.5, 2.5) = ', divmod(7.5, 2.5))
55 print('(2.6, 10.7) = ', divmod(2.6, 0.5))
```

```
(5, 4) = (1, 1)
(10, 16) = (0, 10)
(11, 11) = (1, 0)
(15, 13) = (1, 2)
(8.0, 3) = (2.0, 2.0)
(3, 8.0) = (0.0, 3.0)
(7.5, 2.5) = (3.0, 0.0)
(2.6, 10.7) = (5.0, 0.10000000000000009)
```

Polymorphism In Class methods

- Polymorphism with class methods is useful when we group different objects having the same method.
- We can add them to a list or a tuple, and we don't need to check the object type before calling their methods.
- Python will check object type at runtime and call the correct method



Example

- In the below example, `fuel_type()` and `max_speed()` are the instance methods created in both classes.
- As you can see, we have created two classes `Ferrari` and `BMW`. They have the same instance method names `fuel_type()` and `max_speed()`.
- However, we have not linked both the classes nor have we used inheritance.

```
class Ferrari:
    def fuel_type(self):
        print("Petrol")

    def max_speed(self):
        print("Max speed 350")

class BMW:
    def fuel_type(self):
        print("Diesel")

    def max_speed(self):
        print("Max speed is 240")

ferrari = Ferrari()
bmw = BMW()

# iterate objects of same type
for car in (ferrari, bmw):
    # call methods without checking class of object
    car.fuel_type()
    car.max_speed()
```

```
Petrol
Max speed 350

Diesel
Max speed is 240
```

Polymorphism with Function and Objects

- We can create polymorphism with a function that can take any object as a parameter and execute its method without checking its class type. Using this, we can call object actions using the same function instead of repeating method calls.

Example

```
class Ferrari:
    def fuel_type(self):
        print("Petrol")

    def max_speed(self):
        print("Max speed 350")

class BMW:
    def fuel_type(self):
        print("Diesel")

    def max_speed(self):
        print("Max speed is 240")

# normal function
def car_details(obj):
    obj.fuel_type()
    obj.max_speed()

ferrari = Ferrari()
bmw = BMW()

car_details(ferrari)
car_details(bmw)
```

Polymorphism In Built-in Methods

- The word polymorphism is taken from the Greek words poly (many) and morphism (forms). It means a **method can process objects differently depending on the class type or data type.**
- The built-in function reversed(obj) returns the iterable by reversing the given object.

Example

```
students = ['Emma', 'Jessa', 'Kelly']  
school = 'ABC School'  
  
print('Reverse string')  
for i in reversed('PYnative'):  
    print(i, end=' ')  
  
print('\nReverse list')  
for i in reversed(['Emma', 'Jessa', 'Kelly']):  
    print(i, end=' ')
```

Method Overloading

- The process of calling the same method with different parameters is known as method overloading. Python does not support method overloading.
- Python considers only the latest defined method even if you overload the method. Python will raise a `TypeError` if you overload the method.



Example

```
def addition(a, b):  
    c = a + b  
    print(c)
```

```
def addition(a, b, c):  
    d = a + b + c  
    print(d)
```

```
# the below line shows an error  
# addition(4, 5)
```

```
# This line will call the second product method  
addition(3, 7, 5)
```

Example: User-defined Polymorphic Method

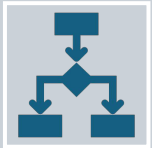
- Let's assume we have an area() method to calculate the area of a square and rectangle. The method will calculate the area depending upon the number of parameters passed to it.
- If one parameter is passed, then the area of a square is calculated
- If two parameters are passed, then the area of a rectangle is calculated.

```
class Shape:
    # function with two default parameters
    def area(self, a, b=0):
        if b > 0:
            print('Area of Rectangle is:', a * b)
        else:
            print('Area of Square is:', a ** 2)

square = Shape()
square.area(5)

rectangle = Shape()
rectangle.area(5, 3)
```

Operator Overloading in Python



Operator overloading means changing the default behavior of an operator depending on the operands (values) that we use. In other words, we can use the same operator for multiple purposes.



For example, the + operator will perform an arithmetic addition operation when used with numbers. Likewise, it will perform concatenation when used with strings.



The operator + is used to carry out different operations for distinct data types. This is one of the most simple occurrences of polymorphism in Python.

Example

```
# add 2 numbers
print(100 + 200)

# concatenate two strings
print('Jess' + 'Roy')

# merger two list
print([10, 20, 30] + ['jessa', 'emma', 'kelly'])
```

300

JessRoy

[10, 20, 30, 'jessa', 'emma', 'kelly']

Overloading + operator for custom objects

- Suppose we have two objects, and we want to add these two objects with a binary + operator.
- However, it will throw an error if we perform addition because the compiler doesn't add two objects.

Example

```
class Book:
    def __init__(self, pages):
        self.pages = pages

    # Overloading + operator with magic method
    def __add__(self, other):
        return self.pages + other.pages

b1 = Book(400)
b2 = Book(300)
print("Total number of pages: ", b1 + b2)
```

```
Total number of pages: 700
```



Overloading the * Operator

- The * operator is used to perform the multiplication. Let's see how to overload it to calculate the salary of an employee for a specific period. Internally * operator is implemented by using the `__mul__()` method.
-

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def __mul__(self, timesheet):
        print('Worked for', timesheet.days, 'days')
        # calculate salary
        return self.salary * timesheet.days

class TimeSheet:
    def __init__(self, name, days):
        self.name = name
        self.days = days

emp = Employee("Jessa", 800)
timesheet = TimeSheet("Jessa", 50)
print("salary is: ", emp * timesheet)
```

Example

Polypormism and Duck Typing



Duck Typing is a concept in Python where the type or class of an object is determined by its behavior (methods and properties) rather than its actual class.



The name comes from the phrase: "If it looks like a duck, swims like a duck, and quacks like a duck, then it probably is a duck."



"In Python, this means that an object is treated based on the methods it implements rather than its actual class type."

Example:

```
1 usage
332 class Duck:
    1 usage (1 dynamic)
333     def quack(self):
334         return "Quack! 🦆"
335
1 usage
336 class Human:
    1 usage (1 dynamic)
337     def quack(self):
338         return "I can quack like a duck! 🦑"
339
340 # Function that expects an object that can quack
2 usages
341 def make_quack(entity):
342     print(entity.quack())
343
344 # Creating instances
345 duck = Duck()
346 human = Human()
347
348 # Using Duck Typing (No need to check type)
349 make_quack(duck)
350 make_quack(human)
```

Example:

```
353 # Parent Class
354 3 usages
354 class Animal:
355     1 usage (1 dynamic)
355     def sound(self):
356         pass # Abstract method (to be overridden)
357
358 # Child Classes
359 1 usage
359 class Dog(Animal):
360     1 usage (1 dynamic)
360     def sound(self):
361         return "Woof!"
362
363 1 usage
363 class Cat(Animal):
364     1 usage (1 dynamic)
364     def sound(self):
365         return "Meow!"
366
367 1 usage
367 class Cow(Animal):
368     1 usage (1 dynamic)
368     def sound(self):
369         return "Moo!"
370
371 # Function demonstrating Polymorphism
372 3 usages
372 def make_sound(animal):
373     print(animal.sound())
374
375 # Creating objects
376 dog = Dog()
377 cat = Cat()
378 cow = Cow()
379
380 # Using polymorphism (same function for different objects)
381 make_sound(dog)
382 make_sound(cat)
383 make_sound(cow)
384
```

Why Use Duck Typing?

Flexible Code → No need to explicitly check object types.

Less Strict Type Checking → More dynamic and Pythonic.

Encourages Interface-Based Programming → Focuses on behavior rather than inheritance.

Benefits of Polymorphism in OOPS:

- **Polymorphism** aids the developer in reusing the program codes. It suggests that the old codes and classes written once, confirmed, and executed can be reused when required.
- This saves a programmer's time.
- The codes and associated classes may be relevant to other methods.
- Only a single variable can be used for storing multiple data types like Float, Int, Long, Double, etc. It simplifies the process of searching and executing the types of variables the users utilize.

